Q. You are given a pre processed dataset using Numpy and Pandas

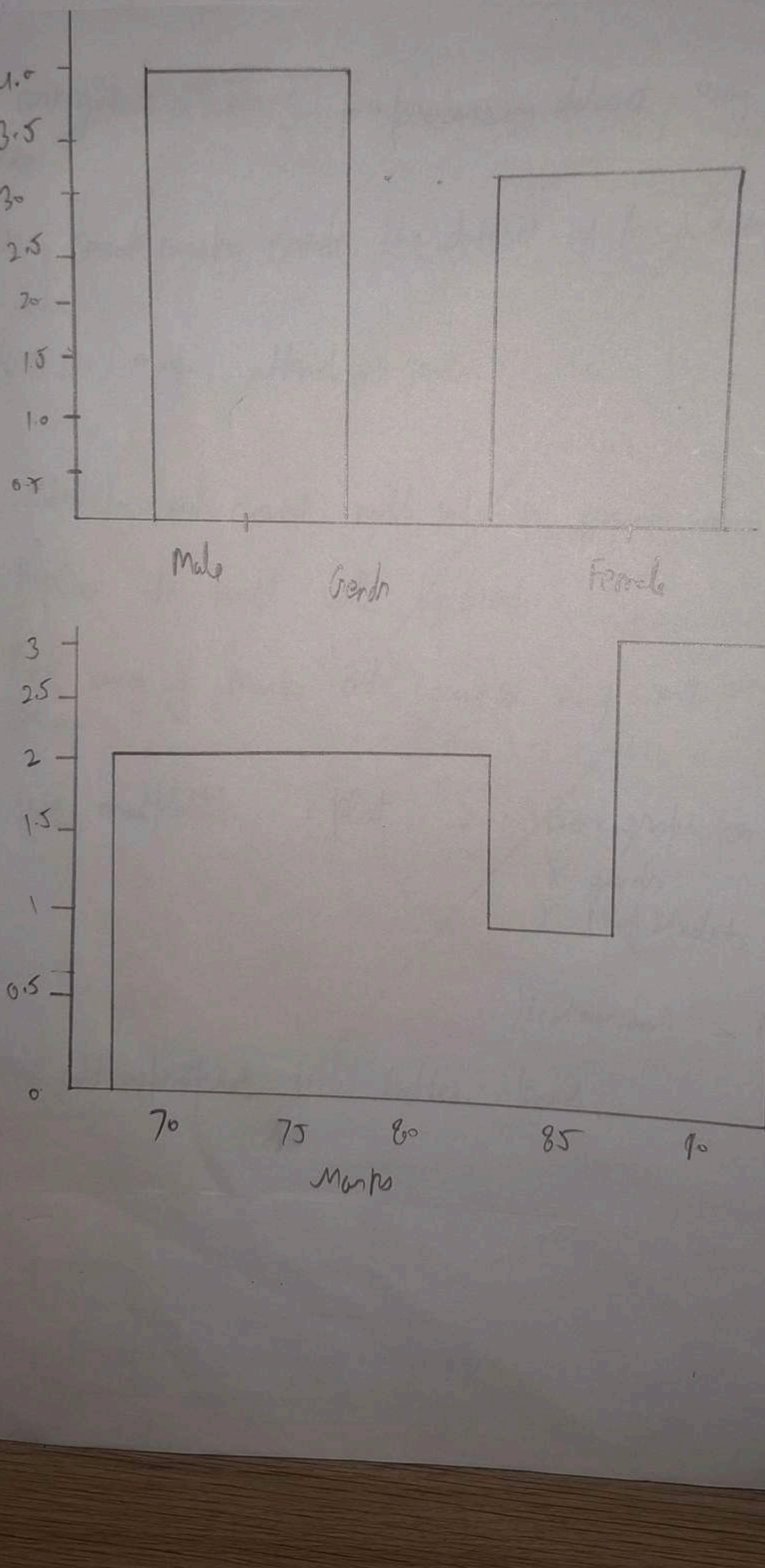
1) Using Numpy, Create the dataset of 10 students of the following details
   - Roll No, marks, attendance, gender.

Q. Identify and count nullvalues in gender

Q. Replace the null values by Mode

Q. Find avg marks and count No of M & F in gender column

Q. Plot graph on gender X = gender
Y = No of students
Histogram → Mark and give
appropriate title, label & legend.


4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
Male
Gender
Female
3
2.5
2
1.5
1
0.5
0
70
75
80
85
90
Marks

```
import numpy as np
roll_no = np.array([1,2,3,4,5,6,7,8,9,10])
marks = np.array([78,85,90,67,88,76,92,81,69,74])
attendance = np.array([90,85,95,80,88,92,96,89,84,86])
gender = np.array(['M','F',None,'M','F',None,'M','F','M',
                   None])

null_count = np.sum(gender==None)
print(null_count)

print(np.mean(marks))

m_count = np.sum(gender=='M')
print(m_count)
f_count = np.sum(gender=='F')
print(f_count)

import matplotlib.pyplot as plt
plt.figure()
plt.bar(['Male','Female'],[m_count,f_count])
plt.xlabel("Gender")
plt.ylabel("No of students")
plt.title("Gender wise distribution of students")
plt.legend(['studnt count'])
plt.show()
```

```
plt.figure()
plt.hist(marks, bins=5)
plt.xlabel("Marks")
plt.ylabel("Number of students")
plt.title("Distribution of Students")
plt.legend(["Marks Frequency"])
plt.show()
```

Q. Generate Marks of 30 students and extend in a list

```
import random
l=[]
for i in range(30):
    l.append(random.randint(0,101))
print(l)
```

Q3. Use the csv files 'medical_records.csv' and 'patient_details.csv'

```
df=pd.read_csv("C:\\Users\\naksh\\Desktop\\AIML\\medical_records.csv")
df2=pd.read_csv("C:\\Users\\naksh\\Desktop\\AIML\\patient_details.csv")
df.head()
```

	record_id	patient_id	diagnosis	BP	Sugar_lvl	visit_cost
0	R101	P013	Hypertension	120/80	140.0	2000
1	R102	P015	Diabetes	120/80	NaN	2500
2	R103	P098	Hypertension	high	140.0	3000
3	R104	P038	Asthma	140/90	NaN	3000
4	R105	P040	Asthma	130/85	NaN	1500

df.tail()

	record_id	patient_id	diagnosis	bp	sugar_level	visit_count
115	R216	P081	Asthma	High	110.0	2.0
116	R217	P033	Diabetes	High	Nan	1.0
117	R218	P063	Hypertension	140/90	Nan	2.0
118	R219	P011	Hypertension	130/85	180.0	2.0
119	R220	P013	Hypertension	high	180.0	1.0

df.info()

df.describe()

df.size → 720

df.ndim → 2

df.shape → (120,6)

print(df2.isnull().mean())

df_filled = df.fillna(0, inplace = True)

print(df_filled)

print(df)

df2['age'] = df2['age'].fillna(df2['age'].mean())

print(df2)

prntdf

	record_id	patient_id	diagnosis	bp	Sugar_lvl	visit_cnt
0	R101	P013	Hypertens...	120/90	140.0	
1	R102	P105	Diabetes	120/80	0.0	
2	R103	P098	Hypertensi...	high	140.0	
118	R219	P011	Hypertens	130/85	180.0	
119	R220	P013	Hypertens	high	180.0	

Q. BFS

```
def bfs(st, goal, graph):
    vis = set()
    queue = [st]
    path = []
    vis.add(st)
    while add(st)
        node = queue.pop(0)
        path.append(node)
        if node = goal:
            return path
        for neighbour in graph[node]:
            if neighbour not in vis:
                vis.add(neighbour)
                queue.append(neighbour)
    return path

graph = { "A" : ['B', 'C'],
          'B' : ['D'],
          'C' : ["E"],
          'D' : []
          'E' : []
}

bfs('A', 'D', graph)
```

['A', 'B', 'C', 'D']

```
def dfs(start, goal, graph, vis=None):
    if vis is None
        vis = set()
    print(start)
    if start == goal:
        return True
    vis.add(start)
    for child in graph[start]:
        if child not in vis:
            if dfs(child, goal, graph, vis):
                return True
    return False

graph = {'A': ['B', 'C'], 'B': ['C', 'A'], 'C': ['A', 'B']}
dfs('A', 'C', graph)
```

A
B
C

09/02/26

Q. Perform Min max normalization on the dataset

```
import pandas as pd
d = pd.read_csv("C:\\Users\\GIET\\Desktop\\P.csv")
d
```

	Roll No.	Name	Height	Weight	Stipend
0	1	Deepak Singh	111	40	11000
1	2	Nitin Rawat	120	48	12000
2	3	Hrithik Singh Rawat	113	53	13000
3	4	utkarsh Chaudhary	127	23	14000
4	5	Tarun Yadav.	114	67	15000

```
c = ["Height", "Weight", "Stipend"]
for n in c:
    d[n] = (d[n] - d[n].min()) / (d[n].max() - d[n].min())
d.to_csv("Pdata.csv", index = False)
```

	Roll No	Name	Height	Weight	Stipend
0	1	Deepak Singh	0	0.375	0.133333
1	2	Nitin Rawat	0.615385	0.475	0.155556
2	3	Hrithik Singh Rawat	0.076941	0.5375	0.177778
3	4	utkarsh Chaudhary	0.205128	0.1625	0.2
4	5	Tarun Yadav.	0.038462	0.7125	0.222222

Q2 What is a graph and why do we use it?

Ans2. A graph is a visual representation of data, consisting of nodes connected by lines to show relationships between variables on a Cartesian plane.

We use graphs to simplify complex data, identify trends, compare values and model networks. making information easy to understand and analyze.

Q3. Types of graph.

Ans3.

1. Bar Graphs → for comparing categories
2. Line Graphs → for trends over time
3. Pie charts → for proportions
4. histogram → for frequency distribution
5. Scatter plots → for relationship
6. Box plot → to identify outliers

Q4. What is bfs and why do we use bfs? Real life applications of bfs (5)

Ans. It is a fundamental graph traversal algorithm that explores nodes level by level, visiting all neighbors of a node before moving to the next level. usually employs a queue data structure.

We use bfs to find the shortest path in unweighted graphs. detect cycles. and traverse data structures efficiently.

Application

1. GPS Navigation System
2. Social Network Sites
3. Network Broadcasts
4. AI and Game development
5. Garbage collection in Programming

05. What is dfs & why we use it. Real life applicati of dfs(5)

As It is a fundamental graph traversal algorithm that explores as deeply possible along each branch before backtracking.

Utility: a stack to manage nodes.

It is used for traversing trees/graphs, detecting cycles, topological sorting and solving pathfinding puzzles.

Memory efficiency, pathfinding in Puzzles, cycle detection, Topological sorting

Real life application.

1. web Crawlers
2. File system Navigation
3. AI Game Solving
4. Puzzle solvers
5 Social Networking → Finding connected groups/friends.

Q Write a program for standard deviation.

```
import pandas as pd
import numpy as np
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
y = np.array([9, 8, 7, 6, 5, 4, 3, 2, 1])
m = (9 * np.sum(x*y) - (np.sum(x) * np.sum(y)))/(9 * np.sum(x*x) - (np.sum(x)
    np**.sum(x))))
c = (np.sum(y) - m * np.sum(x))/9
m, c
```

Output.

np.float64(-1.0), np.float64(10.0))

0 Design and develop a progr to implement the dfs algorithm graph represented in an infollow specification

1. Static graph — 6 nods 10 edges, stored in dictionary

Ab, ac, bd, be, cd, cf, de, df, ef, bf.

```
def dfs(st, goal, graph, vis=None):
    if vis is None:
        vis = set()
    print(st)
    if st == goal:
        return True
    vis.add(st)
    for child in graph[st]:
        if child not in vis:
            if dfs(child, goal, graph, vis):
                return True

graph = {'a':['b','c'], 'b':['d','e','f'], 'c':['d','f'], 'd':['e','f'], 'e':[
    'f'], 'f':[]}

a = dfs('a', 'c', graph)
a
```

output

a
b
d
e
f
c

2. Crack a graph with the number of nodes and edges are provided by user of the function, except the initial and goal of well.

```
graph = { }
n = int(input("enter"))
for i in range(n):
    node = input("enter node")
    graph[node] = []
e = int(input("enter no of edges"))
for i in range(e):
    u = input("enter starting")
    v = input("enter node")
    graph[u].append(v)
    graph[v].append(u)
for key, in graph:
    print(key, ":", graph[key])
initid = input("enter start point")
end = input("enter end point")
n = dfs(initid, end, graph)
n
```

~~Enter~~

Output.

enter start point a

enter end point c

a
b
d
e
c

a. Design and simulate a program to implement the bfs algorithm for graph using AI with following specification:

1. Construct a static graph consisting of 6 nodes, 10 edges, stored using adjacency representation. Ab, aC, bd, be, cd, cf, df, df, ef, bf.

```
def bfs(st, god, graph):
    vis = set()
    queue = [st]
    path = []
    vis.add(st)
    while queue:
        node = queue.pop(0)
        path.append(node)
        if node == god:
            return path
        for neighbor in graph[node]:
            if neighbor not in vis:
                vis.add(neighbor)
                queue.append(neighbor)
    return path.
graph = {'a': ['b', 'c'], 'b': ['d', 'e', 'f'], 'c': ['d', 'f'], 'd': ['e', 'f'], 'e': ['f'],
         'f': []}
n = bfs("a", 'c', graph)
```

X

```
['a', 'b', 'c']
```

2. Create a graph whose nodes and edges are input by the user and accept the initial and goal from user.

```
graph1 = {}
n = int(input("enter no of nodes"))
for i in range(n):
    node = input("enter node")
    graph1[node] = []
e = int(input("enter no of edges"))
for i in range(e):
    u = input("enter starting node")
    v = input("enter ending node")
    graph1[u].append(v)
    graph1[v].append(u)
for key in graph1:
    print(key, ":", graph1[key])
initial = input("enter start node")
final = input("enter final node")
x2 = bfs(initial, final, graph1)
x2.
```

Abhinav
17-02-26

output:

```
enter start node a
enter final node c
['a', 'b', 'c'].
```

Q. Code for applying standard scale to list named data.

Ans def standrd (data):

mean = sum(data) / len(data).

sumsq = 0.

for n in data:

Sum sq t = (n - mean) ** 2.

std = (sumsq / len(data)) ** 0.5.

scaled = []

for n in data:

scaled . append ((n - mean) / std)

return scaled..

data = [10, 20, 30, 40, 50].

print (standrd (data))

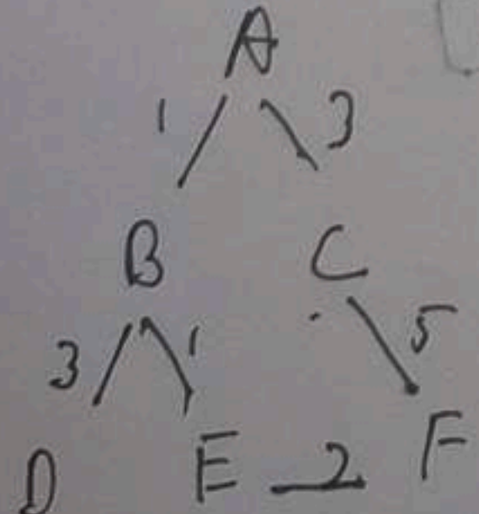
outht.

[-1.414, -0.707, 0.0, 0.707, 1.414]

Q Write a Code to implement astar algorithm with given heuristic values:

A = 6 , B = 4 , C = 4 , D = 2 , E = 2 , F = 0.

Graph



2 Ans

```
def astar(start, goal, graph, ):
    open=[start]
    g={start:0}
    parent={start:None}
    while open:
        n=open[0]
        for i in open
            if g[i]+h[i]<g[n]+h[n]:
                n=i
        if n==goal:
            path=[]
            while n
                path.append(n)
                n=parent[n]
            return path[::-1]
        open.remove(n)
        for m in graph[n]
            if m not in g:
                g[m]=g[n]+graph[n][m]
                parent[m]=n
                open.append(m)
graph={"A":{'B':1,'C':3},
'B':{'D':3,'E':1},'C':{'F':5},'D':{},
'E':{'F':2},'F':{}}
```

```
h = {'A':6, 'B':4, 'C':4, 'D':2, 'E':2, 'F':3}
print(asarray('A', 'F', get(h)))
```

Output
['A', 'B', 'E', 'F']

Q. IQR method code to identify outliers in a given set of data.

Ans. import pandas as pd.

```
data = pd.Series([10, 12, 14, 15, 13, 20, 100])
q1 = data.quantile(0.25)
q3 = data.quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = data[(data < lower) | (data > upper)]
print(f"{outliers.tolist()}")
```

Output
outliers: [100.0]

Q1. Applying Linear Regression model on dataset and Calculate MSE, MAE, RMSE, r_2 Score and plotting the graph of the Best fit line.

Step1. • import numpy as np
• import pandas as pd
• from sklearn.linear_model import LinearRegression
• from sklearn.model_selection import train_test_split
• from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
• import matplotlib.pyplot as plt

```
data = {"x" : [1,2,3,4,5,6,7,8], "y" : [2,4,6,8,10,12,14,16]}
df = pd.DataFrame(data)
df
```

Output:

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10
5	6	12
6	7	14
7	8	16

Step3 `X = df[['x']]`
`Y df['y']`

X_tran, X_Tst, Y_tran, Y_test = tran_test_split(X, Y, test_size
=0.2, rando_stat=42)

Steps = X_tran, X_test, Y_tran, Y_test

Output

C	n
0	1
7	8
2	3
4	5
3	4
6	7
	n
1	2
5	6,
0	2
7	12
2	6
4	10
3	8
6	14

Name: y, dtyt: int64

1	4
5	12

Name: y, dtyt: int64

Step6 model = Liner Regresst()

Step7 model.fit(X_tran, X_tran)

outft:

- Liner Regressio
- Parametes.

Step8: Y_pred = model.predict(X_test)

Step9: Y_pred

out: array([4, 12])

Step10: y_tyst

out:

1	4
5	12

Name: y, dtyt: int64

Step11 slate = mode.coef[0]

interaft = model.interaft_

slate.interate.

out: (1.99999999 ... 5.32907051e-15)

Step12: mean_absalute_error(Y_test, Y_pred)

out: 2.6645352 5910375 7e-15

Step13: mean_squared_error(Y_test, Y_pred)

out: 7.8886090522101180e-30

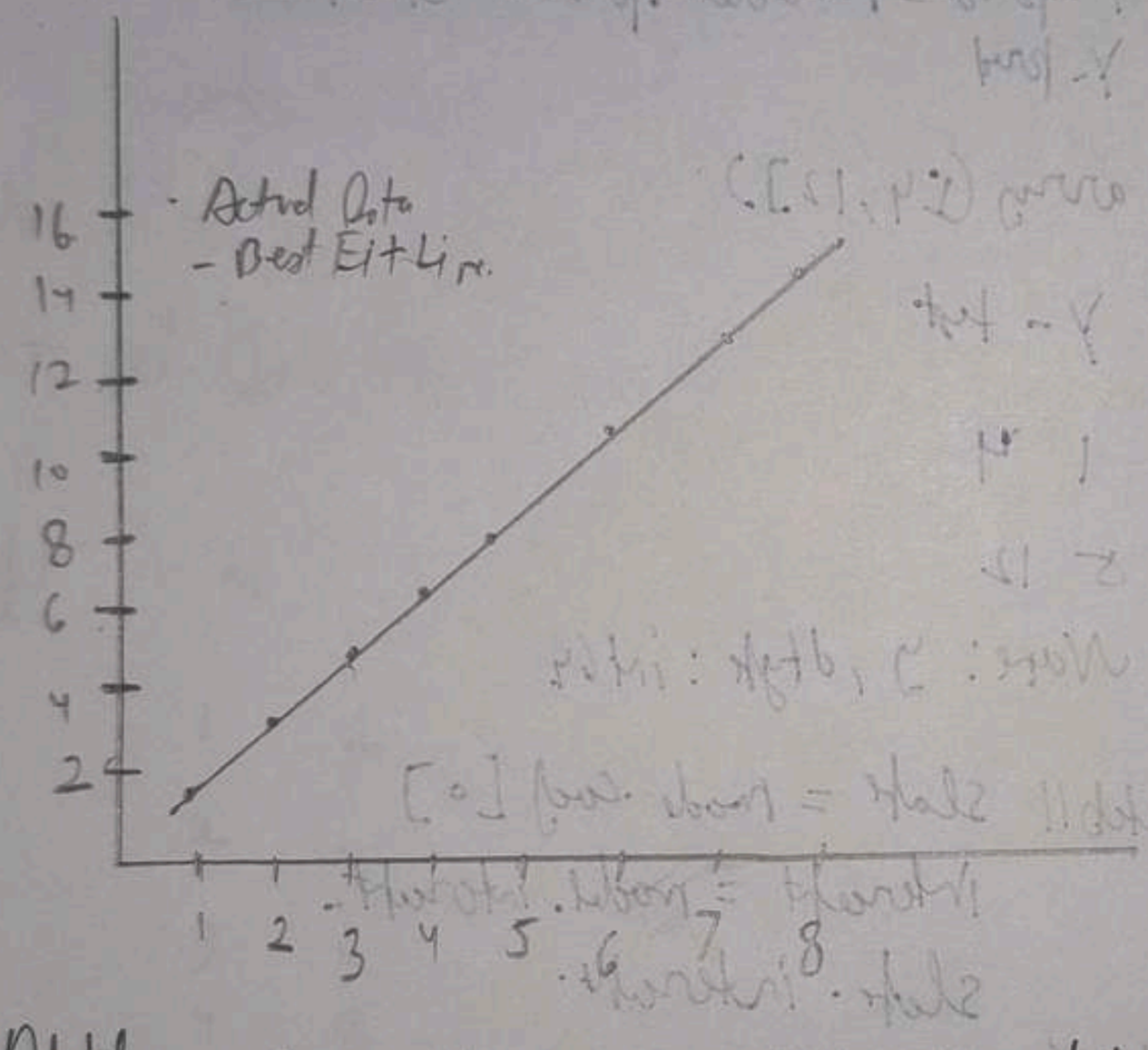
Step14: np.sqrt(mean_squared_error(Y_test, Y_pred))

out: 2.8086677486136 1e-15

Step15: r2_Score(Y_test, Y_pred)

out: 1.0

step:
plt.scatter(X, y, Color="blue", label="Actual Data")
plt.plot(X, model.predict(X), color="red", label="Best Fit Line")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Lin Regression Best Fit Line")
plt.legend()



Q. Applying Linear Regression model on a dataset and calculate MSE, RMSE, MAE, R2 score and plotting the Graph of the Best Fit Line.

Step 1. import numpy as np.
import pandas as pd.
import matplotlib.pyplot as plt.
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

from sklearn.metrics import squared_error, mean_absolute_error, r2_score

Step2. data = { 'x': [11, 12, 13, 14, 15, 16, 17, 18], 'y': [22, 23, 33, 11, 12, 14, 16, 17] }

df = pd.DataFrame(data)

output:

	x	y
0	11	22
1	12	
		33
2	13	33
3	14	11
4	15	12
5	16	16
6	17	16
7	18	17

Step3: x = df[['x']]
Y = df['Y']

Step4 x_train, x_test, y_train, y_test
= train_test_split(x, y, test_size=0.2, random_state=42).

Step5: x_train, x_test, y_train, y_test.

output.

	x
0	11
7	13
2	18
4	15
3	14
6	17.
	2
1	12
5	16

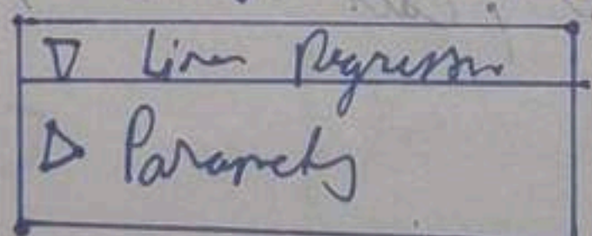
	y
0	22
7	17
2	33
4	12
3	11
6	16

Name: y dtype: int64.

Step6 model = LinearRegression()

Step7 model.fit(x_train, y_train)

out



Step8: y_pred = model.predict(x_test)

Step9: y_pred.

out array([22.1, 16.72])

Step10: y_test

output

1	23
5	14

Name: y, dtype: int64.

Step11: slope = model.coef_[0]
intercept = model.intercept_
slope, intercept

Out (-1.34 ... , 38.3)

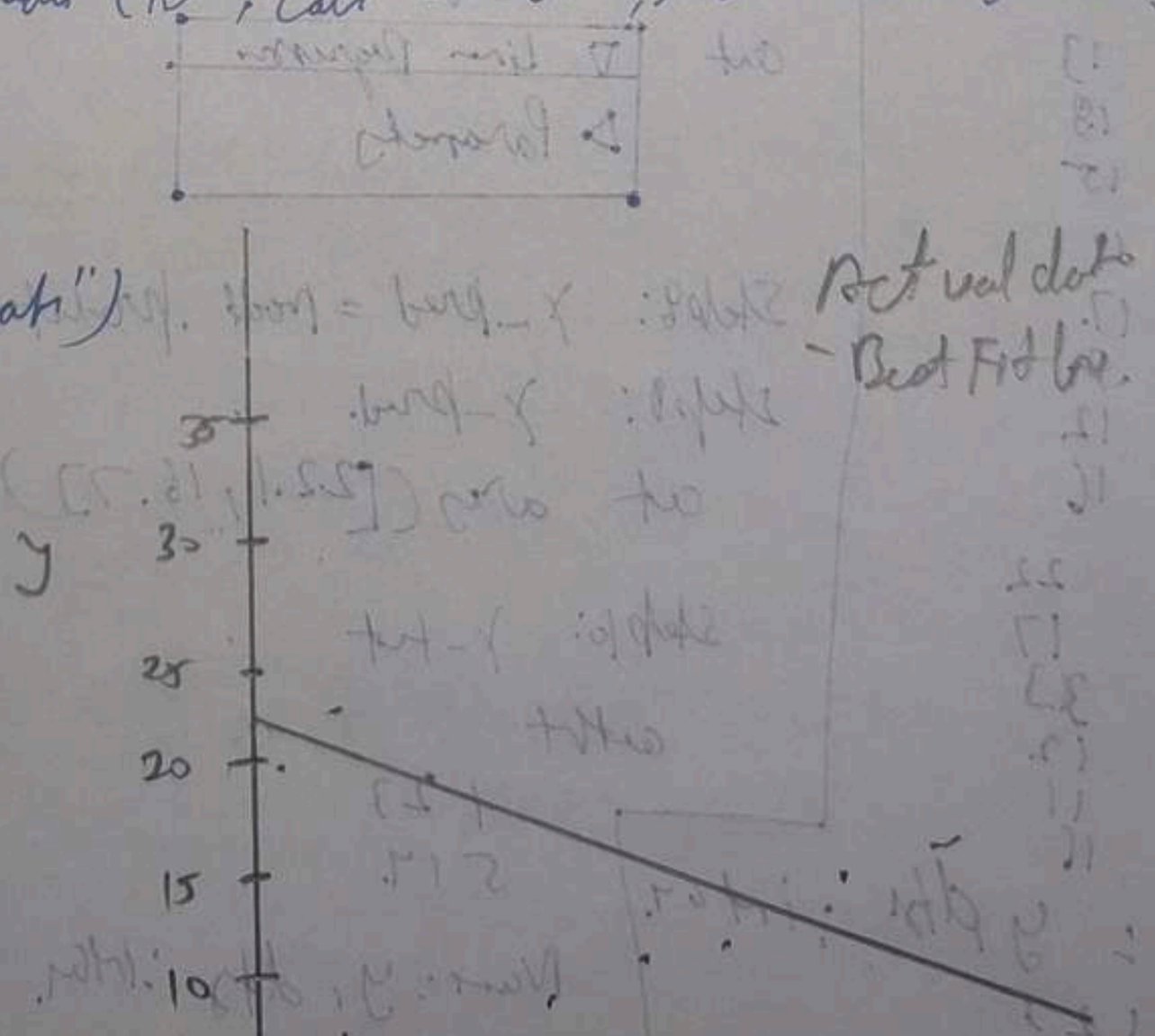
step12: mean_absolute_error(y_test, y_pred)
out : 1.8000007

step13: mean_squared_error(y_test, y_pred)
out : 4.05

step14. np.sqrt(mean_squared_error(y_test, y_pred))
out 2.01246

step15: r2_score(y_test, y_pred)
out 0.8.

step16.
plt.scatter(x, y, color="blue", label="Actual data")
plt.plot(x, model.predict(x), color="red", label="Best fit line")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Best fit line Graph")
plt.legend()
plt.show



Q. Apply linear Regression model a dataset and calculate accuracy_score, precision_score, recall_score, f1 score, confusion matrix.

step1: import numpy as np
from sklearn.neighbors import KNeighborsClassifier.
from sklearn.model_selection import train_test_split.
import pandas as pd.
from sklearn.metrics import accuracy_score, precision_score, recall_score.
confusion_matrix, classification_report, f1_score.

step2: data = {'x': [1,2,3,4,5], 'y': [0,0,1,1,1]}
df = pd.DataFrame(data)
df

output:

	x	y
0	1	0
1	2	0
2	3	1
3	4	1
4	5	1

step3: x = df[['x']]
y = df['y']

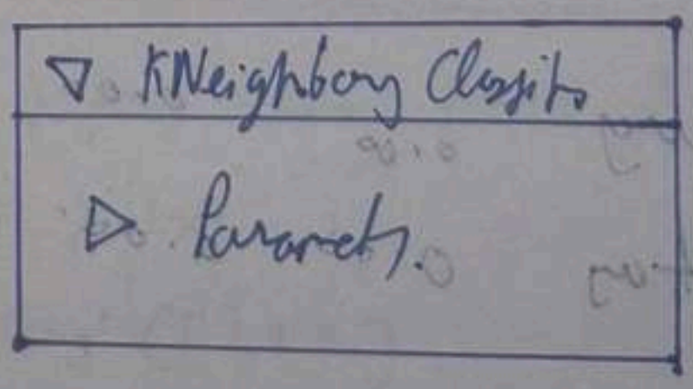
step4:

	x	y
0	1	0
1	2	0
2	3	1
3	4	1
4	5	1

step5: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

step6:
knn = KNeighborsClassifier(n_neighbors=3)

step7: knn.fit(x_train, y_train)



Step 8: y_prd = knn.predict(x_tst)

Step 9: x_tst

y
1
0

Step 10: y_prd

out = array([1])

Step 11: accuracy_score(y_tst, y_prd)

out 0.0

Step 12: precision_score(y_tst, y_prd)

out: 0.0

Step 13 recall_score(y_tst, y_prd)

out: 0.0

Step 14: print(confusion_matrix(y_tst, y_prd))

out [[0 1]
[0 0]]

Step 15: print(classification_report(y_tst, y_prd))

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1.0
1	0.00	0.00	0.00	0.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

04. Calculate accuracy_score, precision_score, recall_score, confusion_matrix, classification_report for a given dataset.

Step 1:
import numpy as np
import pandas as pd

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

Step 2:

data = {"x": [1,2,3,4,5,6,7,8], "y": [2,2,3,2,2,2,3,3]}
df = pd.DataFrame(data)
df.

Out

	x	y
0	1	2
1	2	2
2	3	3
3	4	2
4	5	2
5	6	2
6	7	3
7	8	3

Step 3: X = df[['x']]
y = df['x']

Step 4: X.y

0	1
1	2
2	3
3	4
4	5
5	6
6	7
7	8

0	2
1	2
2	3
3	2
4	2
5	3
6	3
7	

Step 5: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Step 6: knn = KNeighborsClassifier(n_neighbors=3)

Step 7: knn.fit(X_train, y_train)

Out:

KNeighborsClassifier
KNeighborsClassifier()

Step 8: y_pred = knn.predict(X_test)

Step 9: y_test

array([2, 2])

Step 10: accuracy_score(y_test, y_pred)

Out 1.0

Step 12: precision_score(y_test, y_pred)

0.0

accuracy	1.00	1.00	2
macro avg	1.00	1.00	2
weighted avg	1.00	1.00	2

Q. Scaling the data features to a fixed numerical range using min-max method and calculate MAE.

```
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
print("Mean Absolute Error:", mae)
```

```
data = {"X": [11, 12, 13, 14, 15, 16, 17, 18], "y": [23, 43, 53, 63, 73, 83, 93, 103]}
df = pd.DataFrame(data)
```

out

	x	y
0	11	22
1	12	23
2	13	33
3	14	12
4	15	14
5	16	16
6	17	17.
18		

Step 3.
x = df ['x']
y = df ['y'].

Step4: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42).

Step5: x_train, x_test, y_train, y_test

Step6: scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.fit_transform(x_test)

Step7: Scaler = M. MinScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.fit_transform(x_test)

Step 8. model = LinearRegression()
Step 9: model.fit(x_train, y_train)

10: y_pred = model.predict(x_test)

11 y_pred 12 y_test.

13. slope = model.coef_[0].
intercept = model.intercept_
slope.intercept.
27.45000-2).

out (-1.45000-01
step 14. mean_absolute_error(y_test, y_pred)

Q. Apply Logistic Regression

Step 1. import pandas as pd.
import numpy as np
import seaborn as sns, import logistic Regression.
from sklearn.model_selection import train_test_split.
from sklearn.metrics import accuracy_score,
f1_score, confusion_matrix, classification_report.

Step 2:

data = {"study": [1, 2, 3, 4, 5, 6, 7, 8], "Rest": [0, 0, 0, 1, 1, 1, 1, 1]}
df = pd.DataFrame(data).
df

	study	Rest
0	1	0
1	2	0
2	3	0
3	4	0
4	5	1
5	6	1
6	7	1
7	8	1

Step 3: x = df[['study hours']], y = df['Rest']

Step 4: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42).

Step 5: x_train, x_test, y_train, y_test.

```
step6: model = LogisticRegression()
step7: model.fit(X_train, y_train).
step8: y_pred = model.predict(X_test)
step9: y_pred.
output array([0,1,0])
step10) y_test.
output 1 0
       5 1
       0 0
Name: Test, dtype: int64.
step11. accuracy_score(y_test, y_pred)
output 1.0
step12 f1_score(y_test, y_pred).
output 1.0
step13. print(classification_report(y_test, y_pred)).
output
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         3

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4

step14. print(confusion_matrix(y_test, y_pred))
output [[1 0]
        [0 3]].
```

Q K Mens.

Step 1. import pandas as pd.
import numpy as np
import matplotlib.pyplot as plt.
from sklearn.cluster import KMeans.

Step 2: data = {"Value": [1, 2, 3, 10, 11]}
df = pd.DataFrame(data).
df.

output: Value.

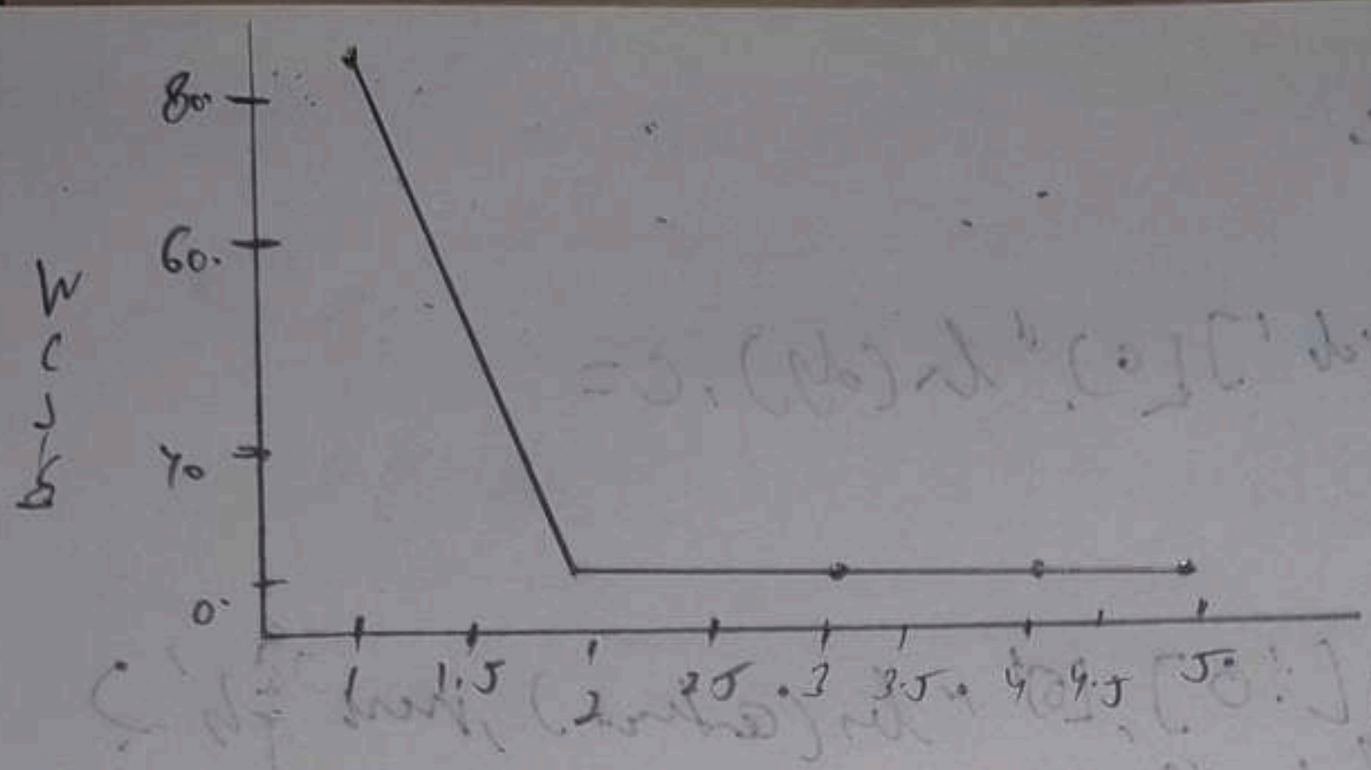
0	1
1	2
2	3
3	10
4	11.

Step 3: X = df[['Value']].values

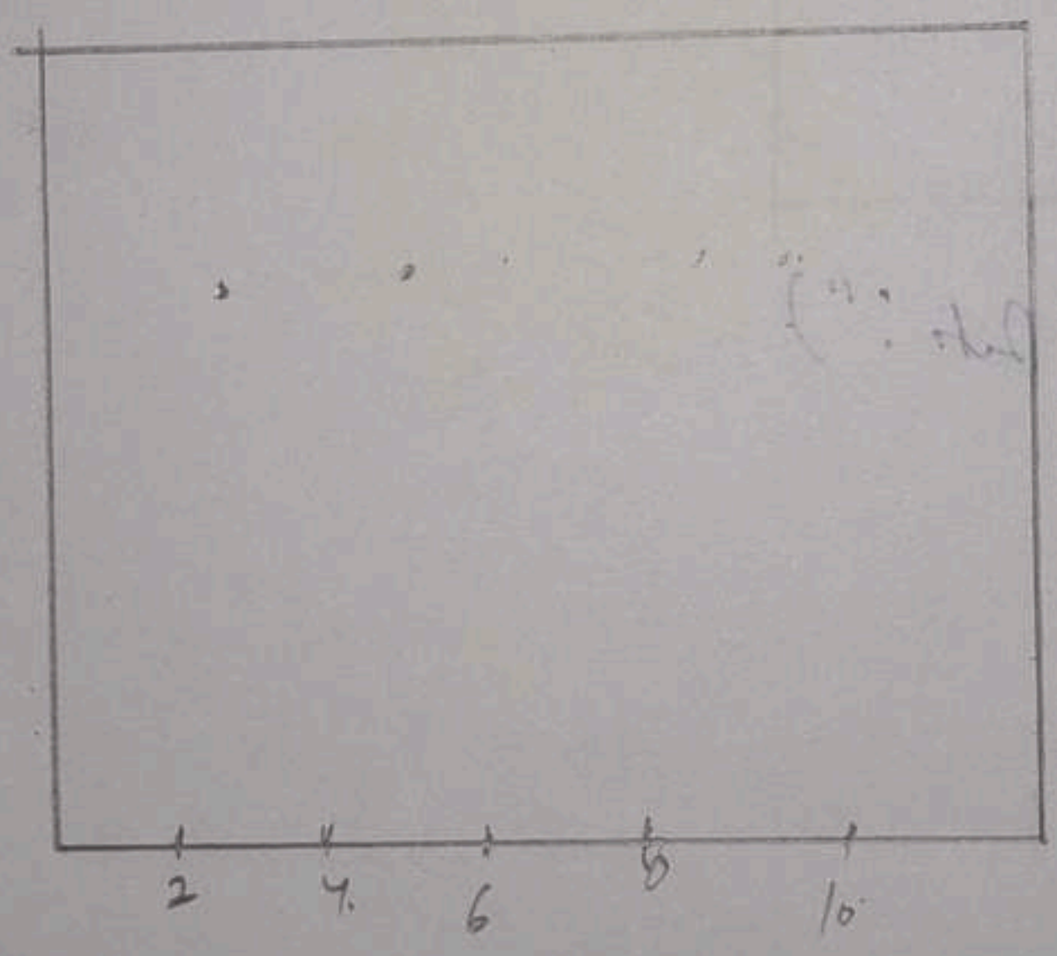
at arms ([[1.], [2.], [3.], [10.], [11.]], dtype, int64).

Step 4. wcss = [].
for i in range (1, 6):
kmeans = KMeans(n_clusters = i, random_state = 42).
kmeans.fit(X).
wcss.append(kmeans.inertia_).

Step 5: plt.plot(range(1, 6), wcss, marker = 'o').
plt.xlabel("No. of clusters").
plt.ylabel("WCSS").
plt.title("Elbow method").
plt.show().



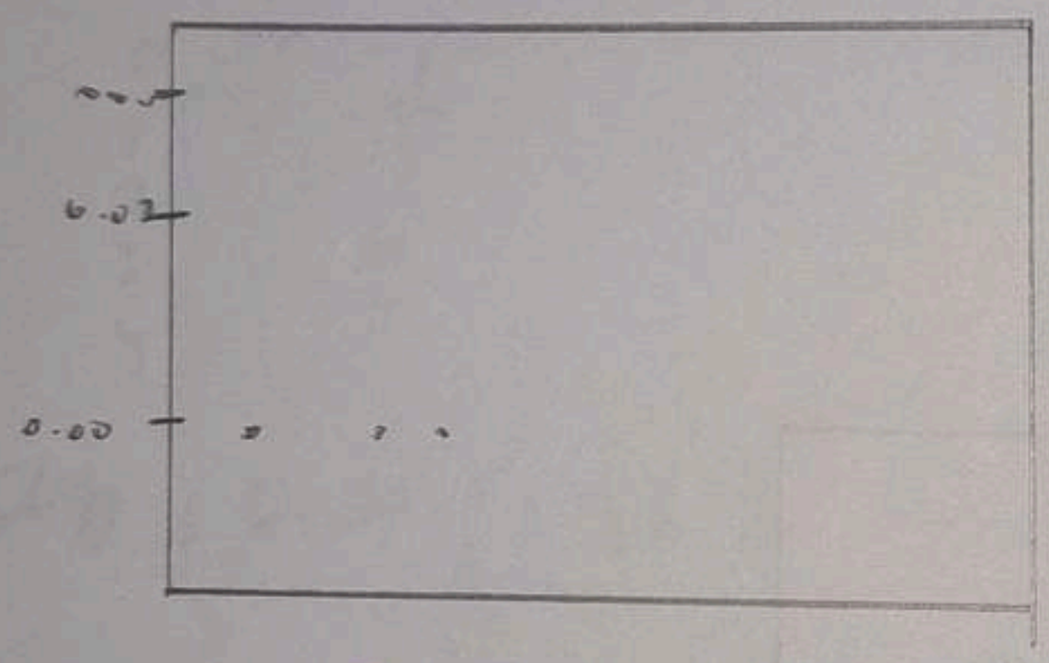
Step 6. plt.figure(1).
plt.scatter(df['val'].[0] * df[df]).
plt.title("Before clust").
plt.xlabel("val").
plt.yticks([]).
plt.show().



Step 7: Kmns = KMeans(n_clust=2, random_state=42).
df['clst'] = Kmns.fit_predict(X).
Centroids = Kmeans.clust_cents_.
step df['clst'].
attr
0 0
1 0
2 0
4 1.

```
New : dist , dist : M ]L
      M. figure ( )
      pt .scatter ( df ['Vol '] [0]." ln (df), c =
          df ['dist '] ).

      pt .scatter (centroid [:0], [0]* ln (centroid), marker ='h')
      pt .title ("After clustering")
      pt .xlabel ('Vol.').
      pt .show()
```




```
std.print ("In clustered Dat :")
    print(df)
clustered dat :.
```

	Vol	Clust.
0	1	0
1	2	
2	3	0
3	10	1
4	11	1